KYUN ROBOTIC ARM UPDATE 29

5/29/2026 9:54 PM

📦 CMD/PowerShell Installation Commands

```
# Create a virtual environment (recommended)
python -m venv kyun_env

# Activate virtual environment
# On Windows:
kyun_env\Scripts\activate
# On Mac/Linux:
# source kyun_env/bin/activate

# Install required packages
pip install flask
pip install flask-socketio
pip install pyserial
pip install python-engineio
pip install python-socketio
pip install eventlet
```

Alternative one-line installation:

```
pip install flask flask-socketio pyserial python-engineio python-socketio eventlet
```

Save to requirements.txt (optional):

```
pip freeze > requirements.txt
# Later install with: pip install -r requirements.txt
```

📋 KYUN AI WEBSERVER - COMPLETE FEATURE LIST

🎮 Control & Interface

- **6-Axis Robotic Arm Control** - Individual motor control via potentiometer knobs
- **Real-time 3D Visualization** - Interactive 3D model with click & drag manipulation

- **PID Motor Control** - Smooth, precise servo movements with adjustable parameters
- **Emergency Abort** - Instant stop button for all operations
- **Global Reset** - Return all servos to home position

👥 Multi-User System (NEW)

- **Editor/Viewer Mode** - Only one editor controls the arm at a time
- **Real-time Sync** - All connected devices see the same 3D model state
- **Role Request System** - Users can request to become editor
- **3D Only Mode** - Edit 3D model without moving physical hardware
- **Sync Assistant** - Help match 3D model to physical arm position

🎤 Voice AI System

- **Voice Recognition** - Control the arm using speech commands
- **Text-to-Speech Responses** - AI responds with voice feedback
- **Custom Voice Commands** - Train your own voice commands
- **Two Tones** - Professional (calm) or Sassy (energetic) personality
- **Neural Status** - Visual indicator of system health

💾 Memory & Sequences

- **12 Memory Slots** - Save and load sequences of poses
- **Teach Pendant** - Record movements in real-time
- **Sequence Execution** - Run saved sequences (with loop option)
- **Smooth Execution** - Fluid transitions between poses (easing functions)
- **Snapshot Capture** - Save current pose to a sequence

⚙️ Hardware Integration

- **Serial Communication** - Connects to Arduino via COM port
- **LED Control** - On/Off/Blink commands
- **Stepper Motor Control** - 0-360 degree absolute positioning
- **Collision Zones** - Set min/max angle limits for each joint
- **Simulation Mode** - Test without hardware connected

🎨 3D Armored Model

- **Futuristic Pink Design** - High-detail 3D model with armor plating
- **Metallic Materials** - Chrome, gold, carbon fiber textures
- **Dynamic Lighting** - Multiple light sources with colored accents

- **Floating Particles** - Atmospheric effects
- **Drag-to-Move** - Click and drag any part of the 3D model

📊 Analytics Dashboard

- **Move Counter** - Total motor movements performed
- **Sequence Counter** - Number of sequences executed
- **Last Action Time** - Timestamp of most recent movement
- **Real-time Updates** - Analytics sync across all devices

💬 Chat Terminal

- **AI Conversation** - Ask questions about developers/team
- **Command Interface** - Send text commands to control the arm
- **Chat History** - Scrollable conversation log
- **Resizable Window** - Drag to resize chat panel

📹 Recording Features

- **Teach Pendant Mode** - Record poses at adjustable intervals
- **Playback Recordings** - Execute recorded sequences
- **Duration Control** - Set hold time for each pose
- **Visual Feedback** - Recording indicator light

🎯 Safety Features

- **Collision Zone Limits** - Prevent mechanical damage
- **Editor-Only Control** - Prevents conflicting commands
- **Hardware Control Toggle** - Disable physical movement while adjusting 3D model
- **Emergency Stop** - Immediate halt all operations

📱 Responsive Design

- **Mobile Support** - Touch-friendly controls
- **Adaptive Layout** - Works on desktop, tablet, and phone
- **Gesture Controls** - Touch and drag potentiometers

🎨 Visual Effects

- **Neon Pink Theme** - Futuristic cyberpunk aesthetic
- **Glass Morphism** - Blur effects and translucent panels
- **Animated Indicators** - Pulsing status lights

- **Audio Visualizer** - Real-time waveform display for voice
- **Custom Scrollbars** - Themed to match design

🔧 Developer Features

- **Custom Commands** - Add personalized voice/button commands
- **Home Position Save** - Save preferred "home" angles for each joint
- **Individual Sync Flags** - Choose which motors sync to hardware
- **Local Storage** - Settings persist between sessions
- **Debug Console** - System log for troubleshooting

🌐 Network Features

- **WebSocket Communication** - Real-time multi-device sync
- **Auto-reconnection** - Handles network interruptions
- **State Recovery** - New clients receive current arm position
- **Cross-device Slot Sync** - Memory units shared across all clients

🎯 Pre-programmed Responses

- Developer recognition (answers questions about team members)
- Context-aware replies
- Professional and Sassy personality modes

🚀 Quick Start

1. **Install dependencies:**

```
pip install flask flask-socketio pyserial
```

2. **Connect Arduino** to COM11 (or modify port in code)
3. **Run the server:**

```
python app.py
```

4. **Open browser** to `http://localhost:5000`
5. **Multiple devices** on same network can connect using your computer's IP address

---

🔧 Hardware Requirements

- Arduino with 6 servos + 1 stepper motor
- Serial communication at 115200 baud
- LED on pin 13 (or as configured)

📝 Note

- Change `COM11` in app.py to match your Arduino port
- All original features remain intact - nothing was removed, only added!